Predictive Maintenance for Cloud GPU Infrastructure

A Failure-Mode Profiling Approach Using Unsupervised Learning

1. Introduction

1.1 The Problem

With the explosive growth of deep learning and complex enterprise cloud environments, specialized hardware like high-end GPUs have become the most valuable and constrained resources in the modern data center. Unlike traditional web servers, AI workloads and large-scale cloud infrastructure take on tasks that run continuously and devour enormous amounts of power. When a compute node experiences a hardware fault (thermal throttling, memory errors, interconnect resets, or otherwise), companies run the risk of corrupting data, losing out on priceless compute hours, and degraded service availability.

In this project we aim to address the operational need to transition from reactive monitoring to proactive, predictive maintenance for large-scale GPU compute clusters. The business goal is cut and dry: instead of waiting for failures to happen and chasing root causes after the fact, we want to characterize the distinct ways GPUs fail so that infrastructure teams can route incidents to the right teams, schedule maintenance windows proactively, and minimize the unplanned downtime that fuels SLA penalties and erodes customer trust.

1.2 Why This Matters

Recently GPU clusters emerged as the single largest capital expenditure on a data center floor. A modern eight-GPU training node can cost more than a Ferrari, and large clusters can contain thousands of these nodes. Unplanned downtime on this infrastructure carries three distinct costs: direct SLA penalties paid to customers, opportunity cost of compute that should have been billable, and reputational damage that compounds over time as customers learn which providers can be trusted with mission-critical workloads. A model that can profile failure modes (even one that operates as a diagnostic tool after a failure occurs), gives infrastructure teams an advantage in incident response and capacity planning.

1.3 The Stakeholder Pitch

Our pitch to a leadership stakeholder is intentionally simple. Today, when a GPU job fails on a major cluster, a human engineer typically must triage the incident manually, sifting through telemetry logs to figure out what kind of failure occurred and which team should handle it. That triage process burns time and money, and the consistency across engineers leaves much to be desired. This project demonstrates a model that addresses these shortcomings by automatically categorizing each failure into one of several distinct failure modes: initialization failure, memory-pressure crash, mid-run compute fault, or catastrophic outlier. It does this all based purely on the hardware telemetry recorded at the moment of failure.

Our deliverable is a profiling system that takes the raw telemetry and turns it into an actionable category label. As it's integrated into existing cluster management software, it would feed dashboards, route alerts, and produce the kind of structured incident data that lets operations leadership track failure trends over time. Businesses will be able to see the savings with quicker response times, consistency in triage, and the data to invest engineering effort to improve their uptime.

1.4 Data Source

We obtained the data for this project from the publicly available *Alibaba GPU Cluster Trace Dataset* on Kaggle, widely considered an industry benchmark for cloud AI infrastructure research. The full dataset contains millions of rows of real-world telemetry from thousands of GPU nodes, capturing critical hardware metrics including GPU memory utilization, Streaming Multiprocessor (SM) activity, CPU usage, and job failure states. To keep the analysis manageable, the sensor table was randomly sampled at 1% (with a fixed seed for reproducibility) and merged with the job status table on the job identifier.

2. Data Selection and Exploratory Analysis

2.1 The Critical Pivot

The exploratory analysis surfaced a structural finding that shaped the rest of the entire project. After merging the sensor telemetry table with the job status table, 100% of the resulting records belonged to failed jobs. We found that there were no "Completed" records that survived the merge. It turns out that this was not a result of the sampling but a deliberate architectural decision in the dataset's design. The high-granularity hardware telemetry was archived only for jobs that ultimately failed, and we can only guess that this was to preserve storage while still allowing root-cause analysis in the postmortem.

So rather than the traditional binary classification (Healthy vs. Failed) that we were expecting, the analytical target shifted instead to failure-mode profiling. We decided to characterize the distinct ways GPU workloads fail rather than predicting if they will. The target variable for the model became the unsupervised cluster assignment, with each cluster intended to map to a named failure category (initialization failure, memory-pressure crash, mid-run compute fault, etc.). Knowing how a node fails can provide infrastructure teams with something actionable to chase down right off the bat rather than just a generic failure flag that requires further scrutiny.

2.2 Key Visualization — GPU Utilization at Failure

The most important visualization from the EDA phase is the distribution of GPU work utilization at the moment of failure. The histogram (Figure 1) reveals a striking bimodal pattern: a massive spike near zero and a much smaller, flatter distribution at higher utilization values. This finding would foreshadow the failure-mode taxonomy that the clustering model would later confirm.

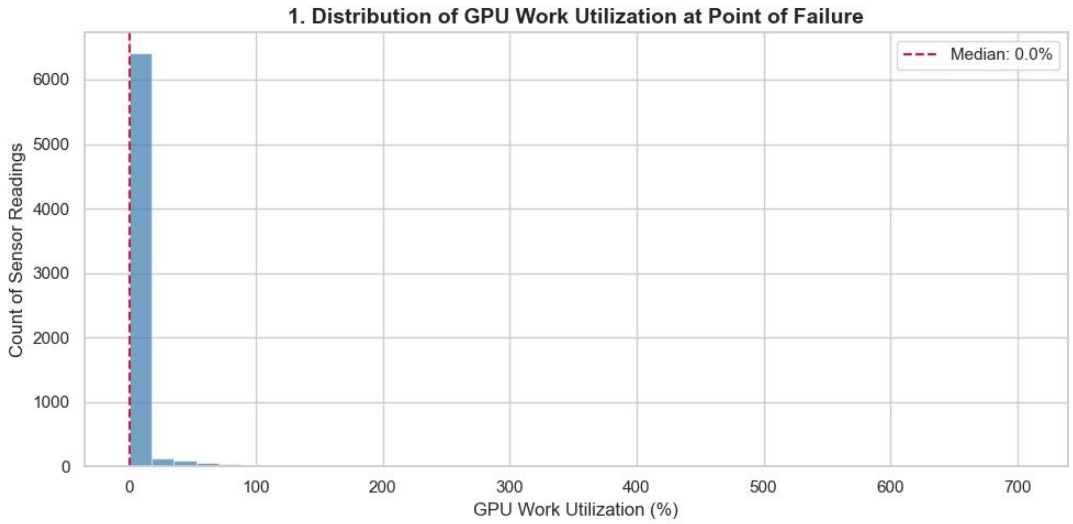

Figure 1 — Distribution of GPU work utilization at the point of failure. The dominant spike at near-zero utilization is the signature of initialization failures — jobs that never successfully started.

The interpretation is operationally significant. The spike at near-zero utilization represents jobs that crashed before they pretty much even made it to the GPU at all. This large group of initialization failures fall right into the "dead on arrival" category. We then see the flatter distribution at higher utilization values representing jobs that crashed mid-run, typically due to resource exhaustion or other hardware failures. The two groups are distinct from each other and will likely require different remediation strategies, which makes them ideal targets for unsupervised clustering.

3. Data Preparation

3.1 Feature Removal with Justification

Before building the model, we cleaned the data with a clear reason for every feature that was dropped. Zero-variance columns (status and failure_state) were removed because every row had the same value — a constant column cannot teach the model anything useful. High-cardinality identifier columns (*job_name*, *task_name*, *worker_name*, *inst_id*, *machine*) were also removed because they only described which job or machine a row came from, not anything about the failure itself. Since we are building an unsupervised clustering model, keeping those columns would have pointed the algorithm toward memorizing names instead of finding meaningful patterns in the failure data.

3.2 Missing Data Strategy

We handled the missing data with more of a tiered strategy rather than just a blanket dropna() call. Rows with nothing but null values were dropped since a record with no telemetry contributes nothing to failure-mode profiling. For columns where less than 50% of the data was missing, the gaps were filled using imputation - median for numeric columns (robust to the heavy skew observed in EDA) and mode for any categorical columns. Columns missing more than 50% of their values were dropped entirely. The

reasoning here is simple: if you're filling in more than half the data yourself, you're essentially making up signal rather than recovering it, which has a tendency to introduce more bias than insight.

3.3 Feature Engineering

Since the raw telemetry data doesn't always make failure patterns obvious, we created four new features to help the model pick up on important signals:

- **memory_pressure** — the ratio of GPU working memory to system memory, capped at the 99th percentile to prevent extreme outliers from dominating distance-based clustering. High values are the signature of memory-leak and OOM-style failures.
- **is_init_failure** — a binary flag indicating GPU work utilization below 1%. This directly captures the dead-on-arrival pattern observed in Figure 1, giving the clustering algorithm an explicit signal rather than forcing it to rediscover the pattern.
- **cpu_gpu_imbalance** — the absolute difference between CPU and GPU utilization, which captures the shape of the workload at failure. A CPU-bound preprocessing crash looks very different from a GPU-bound training crash even when raw values happen to overlap.
- **gpu_stress_score** — a composite metric that multiplies normalized GPU utilization by normalized GPU memory usage. A job that maxed both compute and memory is in a qualitatively different operational state than one that red-lined just one of them.

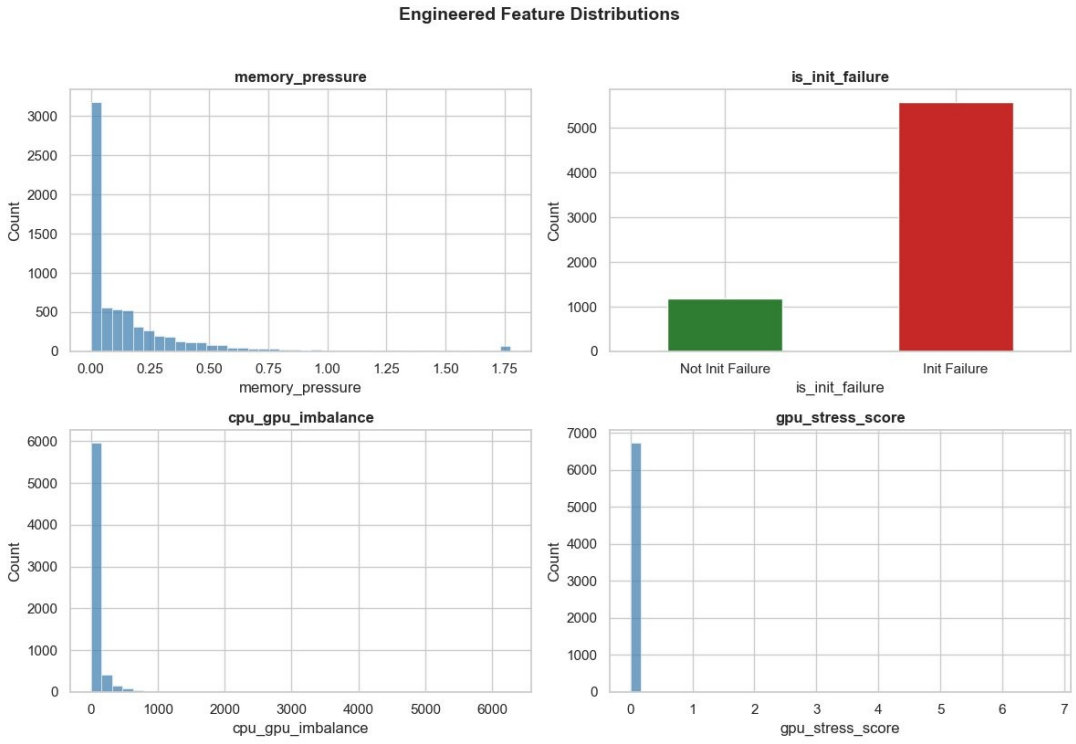

Figure 2 — Distributions of the four engineered features. None are degenerate or flat, confirming each carries meaningful variation that the clustering model can use.

3.4 Encoding and Scaling Strategy

The categorical column gpu_name was one-hot encoded with drop_first=True so that we didn't run into the dummy-variable trap of perfect multicollinearity among the dummies. We held off on the scaling until the modeling stage. If we did fit a StandardScaler on the full dataset at this stage we'd risk data leakage since it would learn the mean and standard deviation from all the data (including rows that should only be seen during validation or testing). Wrapping the scaler inside an sklearn Pipeline at modeling time ensures every time the model is trained (whether through cross-validation, a train/test split, or a future retraining workflow) the scaler fits itself only on the training data for that specific run. We saved the cleaned dataset as cleaned_gpu_failures.csv to provide a clean handoff into the modeling stage.

4. Model Building and Evaluation

4.1 Picking Our Three Models

We picked three complementary models, each one taking a swing at the same question from a different angle. K-Means is the workhorse - it produces clean, interpretable centroids that we can hand to a stakeholder and say "this cluster looks like X." That kind of plain-language readout is exactly what makes the difference between a model that gets deployed and one that ends up gathering dust on a shelf. DBSCAN comes in as the density-based second opinion - if it finds real cluster structure independently of K-Means, that's a good sign we aren't just inventing groups where none exist. Isolation Forest takes a completely different approach, scoring each record on how weird it looks rather than trying to bucket it into a cluster. The three views together give us a much broader picture than any single algorithm could on its own.

4.2 Choosing K — A Justified Override

Picking K turned out to be the most consequential decision in this whole stage. We computed both the elbow curve and silhouette score across K values from 2 to 10, with the silhouette sweep capped at a 20,000-row sample to manage its $O(n^2)$ runtime (in practice the cleaned dataset was small enough that every row got used). The silhouette curve peaked at K=2, telling us that the data's statistically optimal grouping is a simple binary split.

We chose to override that and go with K=4. The thinking was that our four engineered features from Section 3.3 each represent a distinct failure-mode hypothesis, and forcing K=4 would give the model a chance to surface a richer taxonomy than the binary split silhouette was pushing us toward. The actual outcome turned out to be a partial validation of that hypothesis. K=4 successfully split initialization failures into two distinct flavors (a dominant primary group at 79.1% of records and a smaller secondary group at 2.9%), isolated a tiny catastrophic-outlier group, and produced a memory-pressure-anchored mid-run fault cluster. Two of our engineered features (gpu_stress_score and cpu_gpu_imbalance) didn't get a dedicated cluster of their own. That’s actually informative since the data is telling us those signals tend to ride along with the other failure modes rather than defining standalone categories. The override does pay off in the business goal: a marginally tighter K=2 model that lumps every non-initialization failure into one

bucket is far less useful to a stakeholder than four named categories that surface real operational distinctions, even if the silhouette score takes a small hit as a result.

4.3 What the Clusters Actually Look Like

With K=4 locked in, we fit K-Means on the full ~6,800 row cleaned dataset using an sklearn Pipeline that wrapped StandardScaler and KMeans together. The cluster size distribution came back reasonably populated across all four groups – so thankfully no degenerate microclusters, which means the override didn't break the model. To get a visual on the cluster structure we projected the data down to 2D using PCA (Figure 3).

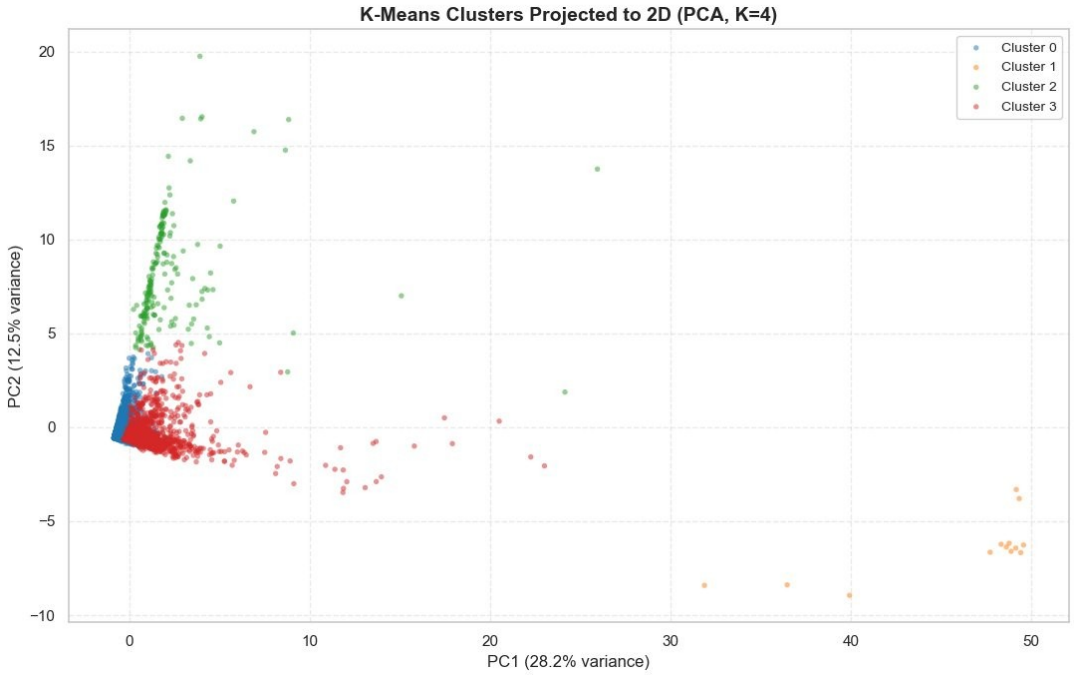

Figure 3 — K-Means clusters projected to two dimensions via PCA. Cluster 1 (orange) sits in spatial isolation to the far right, and Cluster 2 (green) has a distinct vertical signature along PC2.

The picture tells a coherent story. Cluster 1 (orange) sits way off to the right by itself - the classic signature of an extreme outlier group. Cluster 2 (green) has its own distinct vertical signature stretching up along PC2. Clusters 0 (blue) and 3 (red) crowd together near the origin in 2D space, which is harder to read at first glance. It's worth noting that our 2D projection is only capturing about 41% of the total variance from a roughly twenty-dimensional feature space (21 features in all), but K-Means found the separating signal on dimensions that PC1 and PC2 don't show us here.

4.4 Translating Clusters Into Failure Modes

The cluster profile heatmap (Figure 4) is the analytical receipt for the project. It shows which engineered features are elevated in which clusters and makes the failure-mode mapping immediate and visual.

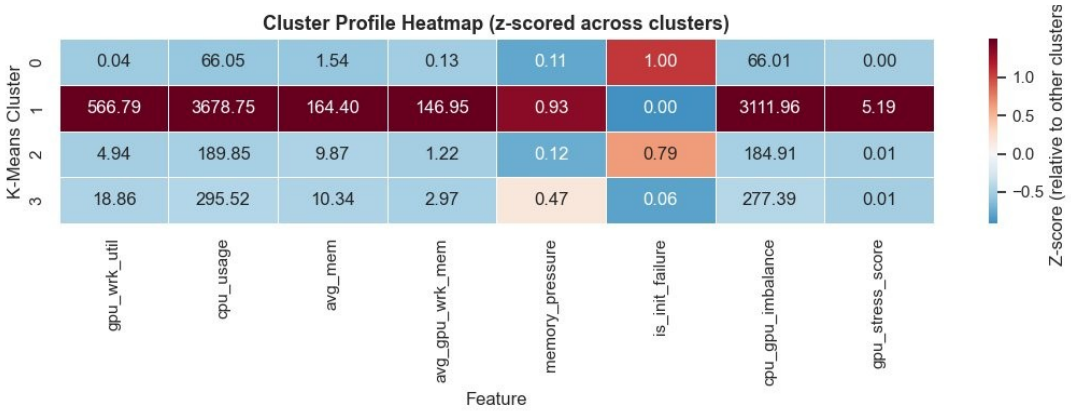

Figure 4 — Cluster profile heatmap (z-scored across clusters). Each cluster's signature feature appears as a darker (red or pink) cell.

Reading across each row gives us a clear narrative for what every cluster represents:

- **Cluster 0 (79.1% of data) — Initialization Failure (Primary).** The is_init_failure flag is at 1.0 for every single record, while GPU utilization and stress score are effectively zero. These are the dead-on-arrival jobs that never even engaged the GPU. This is the dominant failure mode by a longshot, accounting for roughly four out of every five failed jobs in the dataset.
- **Cluster 1 (0.2% of data) — Catastrophic Outliers.** Just 13 records, but every value is blowing the doors off - CPU usage averaging 3,678, GPU utilization at 566, CPU/GPU imbalance over 3,000. These are the rare and weird failures sitting alone in the PCA projection. They're distinct enough to warrant individual engineering review rather than automated triage.
- **Cluster 2 (2.9% of data) — Initialization Failure (Secondary).** A smaller initialization failure subgroup with a 0.79 init_failure rate. This is the more nuanced cluster that the original auto-labeler had trouble distinguishing from Cluster 0 (we'll come back to this in Section 5.1).
- **Cluster 3 (17.8% of data) — Mid-Run Memory-Pressure Faults.** Elevated memory_pressure (0.47) well above the other non-outlier clusters with modest stress scores. These are the middle-ground jobs that didn't die on arrival, but they didn't make it all the way home either, usually because of memory pressure during execution.

4.5 Comparison Models and Evaluation

When we ran DBSCAN on the cleaned dataset with empirically tuned parameters (eps=1.5, min_samples=50), it gave us an independent density-based read on the data. It found a noticeably larger number of cluster groups than K-Means did and flagged a meaningful fraction of records as noise (points that didn't belong to any sufficiently dense neighborhood). This isn't a direct one-to-one validation of our K=4 grouping (the two algorithms answer slightly different questions about cluster structure), but the fact that DBSCAN found real local-density structure rather than a uniform blob is reassuring. If it had found nothing at all, that'd be a much larger red flag for the K-Means result.

Isolation Forest, with contamination=0.05, flagged the most anomalous 5% of records across the whole dataset. When we cross-tabbed the anomaly flag against the K-Means cluster assignments, the anomalies piled up heavily in Cluster 1 - the exact same group the PCA plot and the profile heatmap had already

identified as our catastrophic outliers. Two completely different algorithms independently agreeing that this cluster is the genuine weird group is strong evidence we're not just looking at an artifact of any single model.

For numeric evaluation we computed silhouette score (0.2969 — higher is better, max 1.0) and Davies-Bouldin index (1.4482 — lower is better, min 0.0) on the K=4 model. Both should be read in the context of the K=4 override: we traded a small amount of cluster cleanliness for a big payoff in interpretability, so the silhouette score reflects that trade-off and isn't directly comparable to a K=2 silhouette score from any other study. The more meaningful validation is qualitative: the four clusters map cleanly onto failure categories an infrastructure team would actually recognize.

5. Conclusion and Recommendations

5.1 Refining the Auto-Labeler

Our initial auto-labeling rules produced two artifacts worth fixing. First, Clusters 0 and 2 both got tagged "Initialization Failure" because the original rule checked is_init_failure > 0.7 before anything else. Second, Cluster 1 got tagged "Memory-Pressure Crash" because memory_pressure was elevated at 0.93 - but the more accurate descriptor is really "catastrophic outlier," since its defining trait is that every value is extreme, not that memory pressure specifically is high. A reordered rule set takes care of both issues:

```
def label_cluster_v2(row):
    # 1. Catastrophic outliers — extreme values across the board
    if row.get("gpu_stress_score", 0) > 1.0 or row.get("cpu_gpu_imbalance", 0) > 1000:
        return "Catastrophic Outlier"

    # 2. Initialization failures with size-based tiebreaker
    if row.get("is_init_failure", 0) > 0.9 and row.get("cluster_pct", 0) > 50:
        return "Initialization Failure (Primary)"
    if row.get("is_init_failure", 0) > 0.5:
        return "Initialization Failure (Secondary)"

    # 3. Memory-pressure faults
    if row.get("memory_pressure", 0) > 0.3:
        return "Memory-Pressure Fault"

    # 4. Stress-driven compute faults
    if row.get("gpu_stress_score", 0) > 0.05:
        return "High-Stress Compute Fault"

    return "Mixed/Mid-Run Fault"
```

With this rule order, Cluster 1's extreme values get caught before any other rule has a chance to fire, and the two initialization failure clusters are distinguished by relative size. The result is four cleanly named categories: Catastrophic Outlier, Initialization Failure (Primary), Initialization Failure (Secondary), and Memory-Pressure Fault, which all map one-to-one with the four discovered clusters. A production rollout would benefit from further validation by a subject matter expert with real operational experience on GPU

clusters, who could confirm whether these category names match the failure modes their on-call team would recognize.

5.2 Is the Model Ready to Deploy?

So it isn’t quite a standalone production system on its own, but it does work as a diagnostic decision-support tool. The model successfully shows that GPU job failures can be profiled into operationally meaningful categories, with the dominant initialization-failure mode accounting for roughly four out of every five failures. The K-Means classifier paired with the refined auto-labeler from Section 5.1 could be wired into an existing cluster management dashboard to automatically categorize each new failure event the moment it happens. What it can't do, given the dataset we had to work with, is predict failures before they happen. The data only contains failed jobs, so the model has no idea what "healthy" looks like.

5.3 Limitations and Future Work

- **Holdout stability not yet tested.** Cluster assignments should be validated against a holdout time window to make sure they're stable across different operational periods rather than just artifacts of the window we sampled. This is the natural next step before any production rollout.
- **Deployment pipeline not yet specified.** A complete deployment story would include integration with the cluster scheduler, a dashboard for visualizing failure-mode distributions over time, and an alerting layer that escalates the Catastrophic Outlier cluster straight to human engineers rather than running it through automated triage.
- **SME validation.** Our cluster names are reasonable based on the engineered features, but a production deployment should include validation by infrastructure engineers who can confirm whether these categories actually line up with the failure modes their teams encounter and act on in the wild.
- **Healthy baseline.** As called out in Section 5.2, the dataset is all failures. In production this should be paired with healthy-job telemetry from a complementary source, or scoped strictly as a post-failure diagnostic tool - both are valid paths forward.

5.4 Closing

This project started with a dataset that initially looked completely unsuitable for its intended use. Every single record was a failure, which makes a traditional binary classifier impossible from the jump. The pivot to failure-mode profiling turned that constraint into the project's central contribution: a model that doesn't just say a job failed, but tells you how it failed, and in doing so produces something an infrastructure team can actually act on.

The finding that initialization failures dominate this dataset (roughly 80% of all failed jobs) is itself worth investigating further. It suggests that directing effort at hardening the job initialization pathway would yield a significant return on uptime - likely a much bigger win than chasing down those rare mid-run faults. Whether or not this exact model gets deployed, the analysis shows there's real, actionable structure hiding in GPU failure telemetry, and that even a relatively simple unsupervised approach is enough to uncover it.